



CSCE 633-700: Machine Learning

Neural Networks (Part 1)

Overview (6)

- Perceptron
 - Representation
 - Learning
 - Examples
- Multilayer Perceptron
 - Representation
 - Learning: Backpropagation
 - Practical issues

Backpropagation (1)

Multilayer Perceptron: Representation

- **Input:** $\mathbf{x} \in \mathbb{R}^D$
- **Output:**
 $y \in \{0, 1\}$ or $y \in \{1, \dots, K\}$ (classification)
 $y \in \mathbb{R}$ or $y \in \mathbb{R}^K$ (regression)
- **Training data:** $\mathcal{D}^{train} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$
- **Model:** $h_{\mathbf{W}, \mathbf{b}}(\mathbf{x})$
represented through forward propagation (see previous slides)
- **Model parameters:** weights $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}$ and biases $\mathbf{b}^{(1)}, \dots, \mathbf{b}^{(L)}$

Multilayer Perceptron: Evaluation criterion $J(\mathbf{W}, \mathbf{b}, \mathcal{D}^{train}) = \frac{1}{2} \|h_{\mathbf{W}, \mathbf{b}}(\mathbf{x}) - y\|_2^2$ (regression)

$J(\mathbf{W}, \mathbf{b}, \mathcal{D}^{train}) = y \log h_{\mathbf{W}, \mathbf{b}}(\mathbf{x}) + (1 - y) \log(1 - h_{\mathbf{W}, \mathbf{b}}(\mathbf{x}))$ (classification)

Backpropagation (2)

Multilayer Perceptron: Evaluation criterion

Regression

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^M \frac{1}{2} \|h_{\mathbf{W}, \mathbf{b}}(\mathbf{x}_n) - y_n\|_2^2 + \frac{\lambda}{2} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (W_{ji}^{(l)})^2$$

s_l : # nodes in l^{th} layer

Classification

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^M (y_n \log h_{\mathbf{W}, \mathbf{b}}(\mathbf{x}_n) + (1 - y_n) \log(1 - h_{\mathbf{W}, \mathbf{b}}(\mathbf{x}_n))) \\ + \frac{\lambda}{2} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (W_{ji}^{(l)})^2$$

We will perform **gradient descent**

Backpropagation (3)

Gradient descent for regression

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^M \frac{1}{2} \|h_{\mathbf{W}, \mathbf{b}}(\mathbf{x}_n) - y_n\|_2^2 + \frac{\lambda}{2} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (W_{ji}^{(l)})^2$$

$$W_{ij}^{(l)} := W_{ij}^{(l)} - \alpha \frac{\partial J(\mathbf{W}, \mathbf{b})}{\partial W_{ij}^{(l)}}$$

$$b_i^{(l)} := b_i^{(l)} - \alpha \frac{\partial J(\mathbf{W}, \mathbf{b})}{\partial b_i^{(l)}}$$

Note: Initialize the parameters randomly → **symmetry breaking**

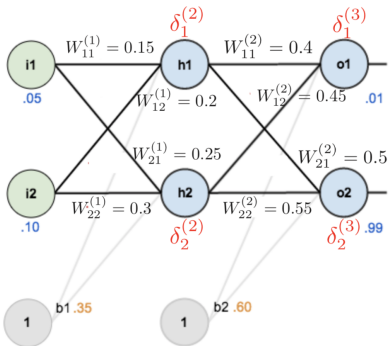
Use **backpropagation** to compute partial derivatives $\frac{\partial J(\mathbf{W}, \mathbf{b})}{\partial W_{ij}^{(l)}}$ and $\frac{\partial J(\mathbf{W}, \mathbf{b})}{\partial b_i^{(l)}}$

Backpropagation (4)

Intuition

- Given a training example $(\mathbf{x}_n, y_n)$, we run a "forward pass" to compute all the activations
- For each node i in layer l , we compute an **error term** $\delta_i^{(l)}$ that measures how much that node was "responsible" for any errors in the output
 - Output node: difference between activation and target value
 - Hidden nodes: weighted average of the error terms of the nodes from the previous layer (i.e. $l + 1$)

Backpropagation (5)



Backpropagation Implementation

- For each node i in output layer L
 - $\delta_i^{(L)} = (\alpha_i^{(L)} - y_n) f'(z_i^{(L)})$
- For each node i in layer $l = L - 1, L - 2, \dots, 2$
 - Hidden nodes: $\delta_i^{(l)} = \left(\sum_{j=1}^{s_{l+1}} W_{ji}^{(l)} \delta_j^{(l+1)} \right) f'(z_i^{(l)})$
- Compute the desired partial derivatives as:
 - $\frac{\partial J(\mathbf{W}, \mathbf{b})}{\partial W_{ij}^{(l)}} = \alpha_j^{(l)} \delta_i^{(l+1)}$
 - $\frac{\partial J(\mathbf{W}, \mathbf{b})}{\partial b_i^{(l)}} = \delta_i^{(l+1)}$
- Update the weights as:
 - $W_{ij}^{(l)} := W_{ij}^{(l)} - \alpha \frac{\partial J(\mathbf{W}, \mathbf{b})}{\partial W_{ij}^{(l)}}$
 - $b_i^{(l)} := b_i^{(l)} - \alpha \frac{\partial J(\mathbf{W}, \mathbf{b})}{\partial b_i^{(l)}}$

[Detailed solution of example in Handouts]